

```
!pip install -q librosa soundfile numpy scipy matplotlib
```

```
import numpy as np
import librosa, librosa.display
import matplotlib.pyplot as plt
import soundfile as sf
import scipy.signal as sig
from IPython.display import Audio, display

eps = 1e-10
```

```
import requests

# The previous URL was returning a 403 Forbidden error.
# Using a different publicly accessible audio URL.
audio_url = "https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3"
local_audio_path = "audio.mp3"

# Download the audio file locally
response = requests.get(audio_url)
response.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)
with open(local_audio_path, 'wb') as f:
    f.write(response.content)

x, sr = librosa.load(local_audio_path, sr=22050, mono=True)

print("Audio Loaded:", len(x)/sr, "seconds")
display(Audio(x, rate=sr))
```

Audio Loaded: 372.7151020408163 seconds

0:00 / 6:12

```
def ir_delta():
    h = np.zeros(1); h[0] = 1
    return h

def ir_lowpass(length=2048, cutoff=3000):
    t = np.arange(length) - length//2
    h = np.sinc(2*cutoff/22050 * t) * np.hamming(length)
    h = h / (np.sum(h)+eps)
    return h

def ir_small_reverb(length=1500, decay=0.8):
    t = np.arange(length)
    env = np.exp(-t/(length/5))
    noise = np.random.randn(length) * 0.02
    h = env * noise
    h = h / (np.max(np.abs(h))+eps)
    return h

IR_LIST = {
    "delta": ir_delta(),
    "lowpass": ir_lowpass(),
    "small_reverb": ir_small_reverb()
}
```

```
def inverse_filter(y, h, alpha=1e-3):
    N = len(y) + len(h)
    Nfft = 2*int(np.ceil(np.log2(N)))

    H = np.fft.rfft(h, Nfft)
    Y = np.fft.rfft(y, Nfft)

    G = np.conj(H) / (np.abs(H)**2 + alpha)
    Xhat = np.fft.irfft(Y * G, Nfft)

    return Xhat[:len(x)]
```

```
def plot_spec(sig, title):
    S = librosa.stft(sig)
    S_db = librosa.amplitude_to_db(np.abs(S) + eps)
    plt.figure(figsize=(10,3))
    librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='hz')
    plt.title(title)
    plt.colorbar()
    plt.show()
```

```
for name, h in IR_LIST.items():
    print("\n=====")
    print("IMPULSE RESPONSE:", name)
    print("=====")

    # Convolution
    y = sig.fftconvolve(x, h, mode="same")

    # Inverse filtering
    x_hat = inverse_filter(y, h, alpha=1e-3)

    # Compute SNR
    snr = 10*np.log10(np.sum(x**2)/(np.sum((x-x_hat)**2)+eps))
    print("SNR:", snr, "dB")

    # PLOTS
    plot_spec(y, f"Convolved ({name})")
    plot_spec(x_hat, f"Inverse Filter Result ({name})")

    # AUDIO
    print("Convolved Audio:")
    display(Audio(y, rate=sr))

    print("Inverse Filtered Audio:")
    display(Audio(x_hat, rate=sr))
```

